

CSE4006

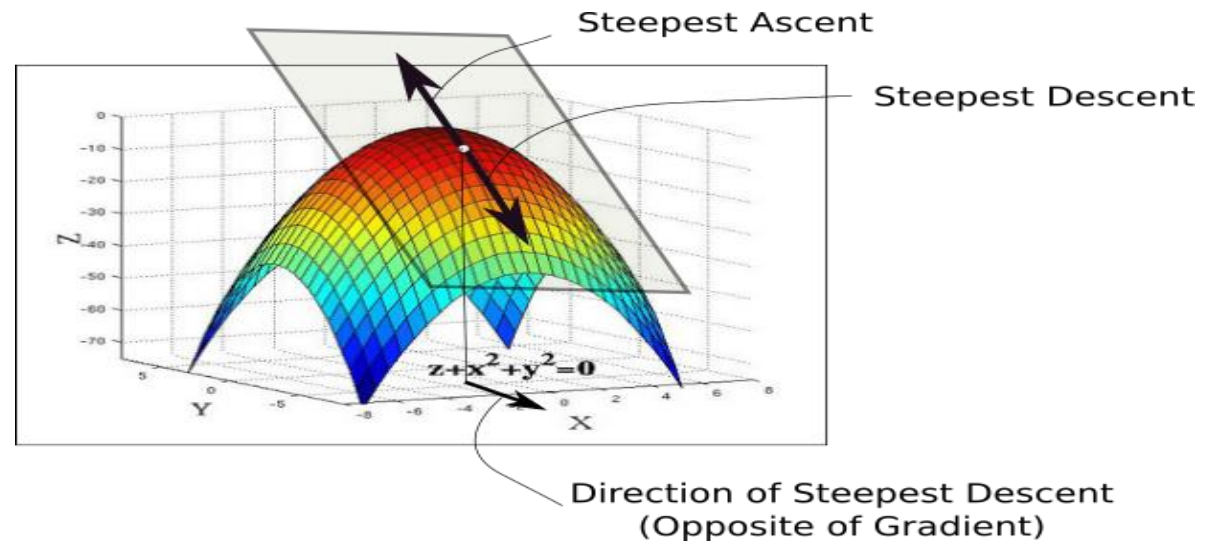
DEEP LEARNING

Gradient based optimization

Linear Algebra-Scalars, Vectors, Matrices and Tensors-
Operations –Special kind of Matrices and Vectors-
Probability Basics-Gradient based optimization-
Linear Classifiers, Linear Machines with Hinge Loss-
Introduction to Neural Network-Activation functions-
Tensorflow , keras operations

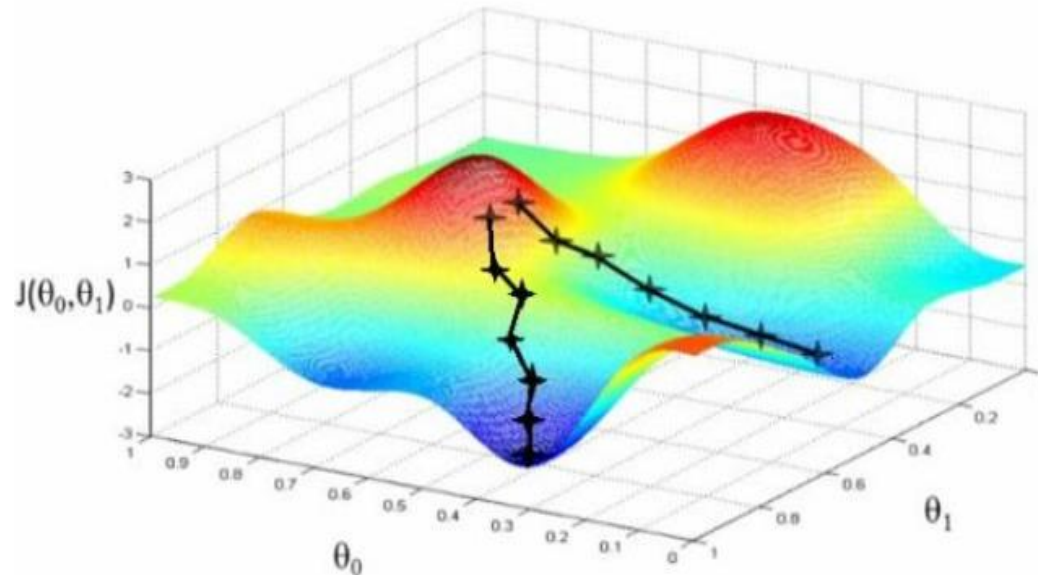
Gradient-Based Optimization

- Optimization is the task of either minimizing or maximizing the function $f(x)$ by varying x .
- Optimizing the problems by *minimizing* $f(x)$.
- Maximization may be accomplished via a minimization algorithm by minimizing $-f(x)$
- The function to be minimized **or** maximized is the **objective function** or **criterion**.
- If it is minimization function, call it as the **cost function, loss function, or error function**
- Value that minimizes or maximizes a function is denoted with a superscript i.e., $x' = \arg \min f(x)$.



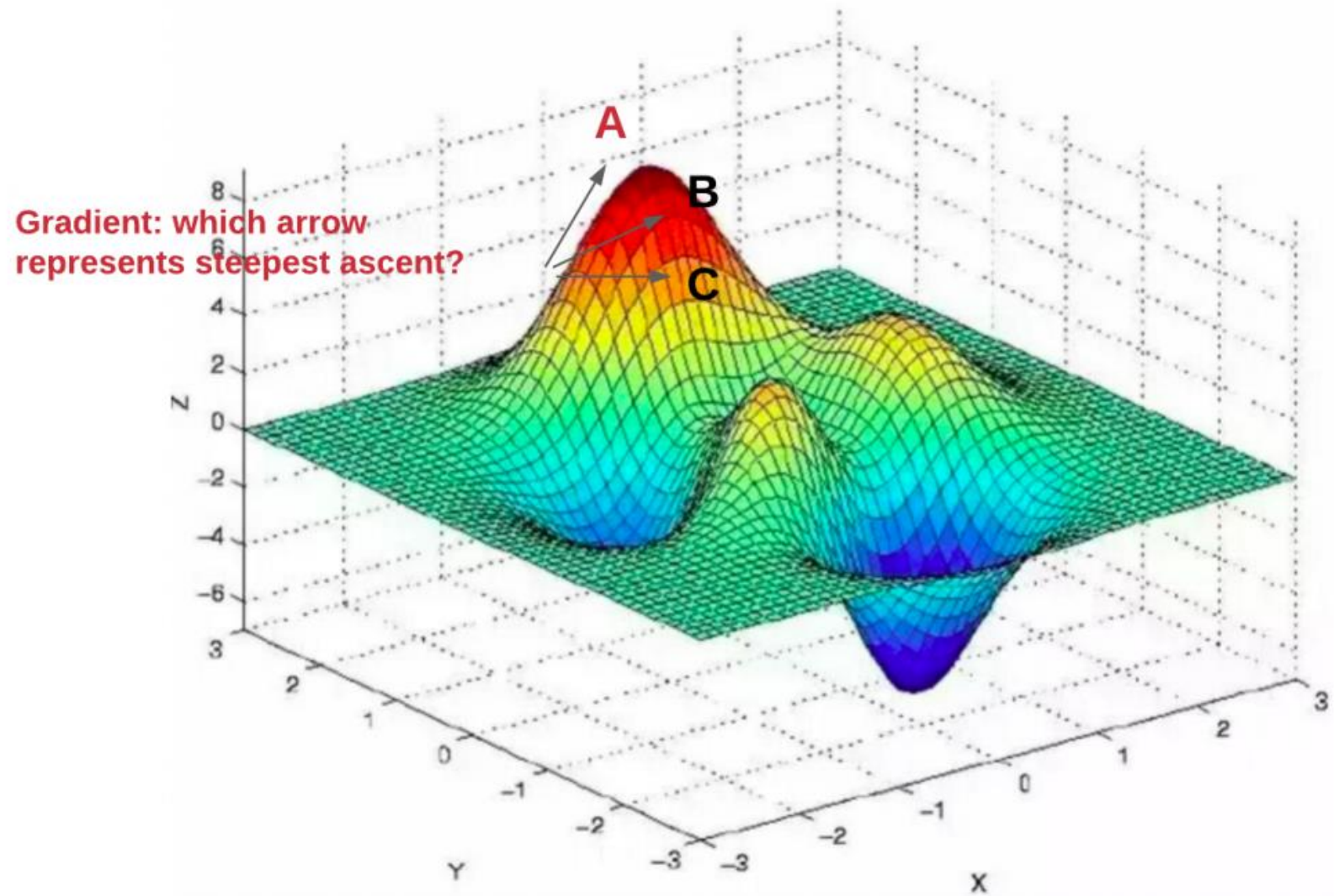
Gradient descent Optimization

- Gradient descent is first-order iterative optimization process for finding a local minimum of a differentiable function.
- Take steps proportional to the **negative** of the gradient (move away from the gradient in the direction of **steepest decrease** or **descent**) of the function at the current point.



Gradient ascent Optimization

- Gradient ascent of a function at any point represent the direction of **steepest increase** or **ascent** of function at that point.
- Take steps proportional to the **positive** of the gradient (moving towards the gradient), to approach a local maximum of the function.



Example : Linear model

	x_1	x_2	y	y'
S_1	0	0	0	0
S_2	0	1	1	1
S_3	1	0	1	1
S_4	1	1	1	1

$$Y' = w_1 * x_1 + w_2 * x_2 + \dots + w_n * x_n + b = W^T X + b$$

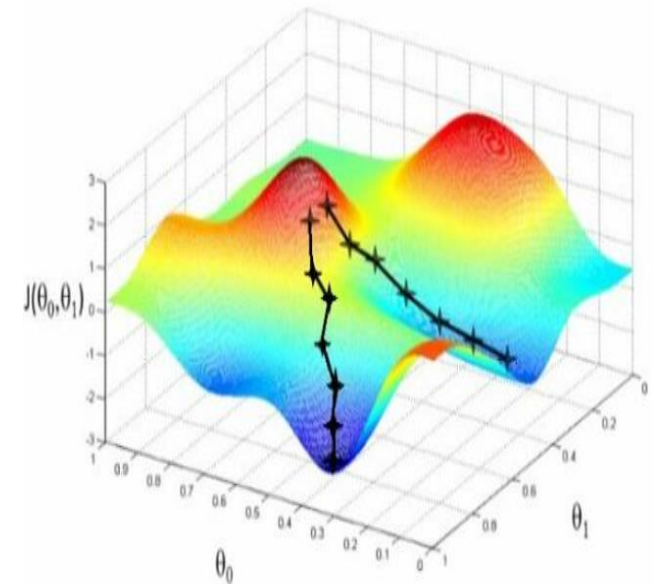
- Error = Y' (Predicted) - Y (Actual)
- A model always needs low error with maximum accuracy
- Minimize the error by push the pointer to minima using alpha (learning rate).

Steps to implement Gradient Descent algorithm

- Randomly initialize values.
- Update values.

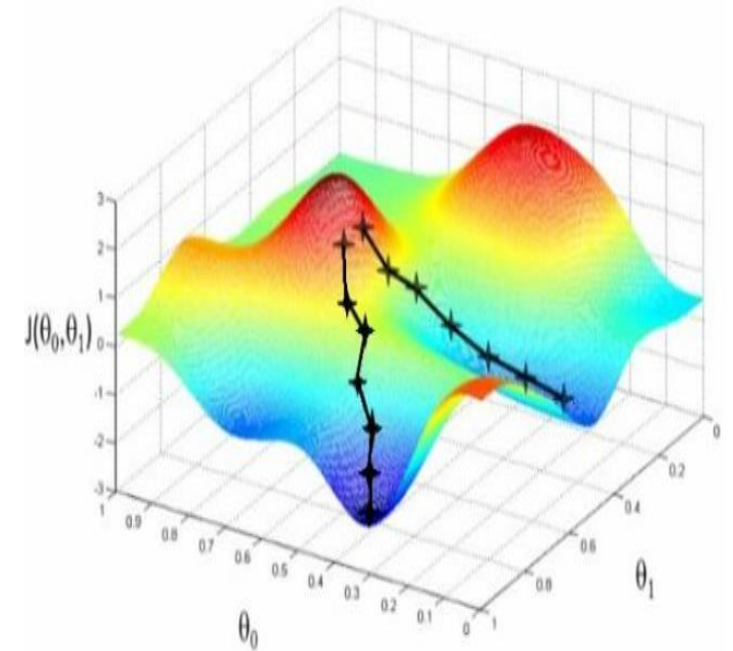
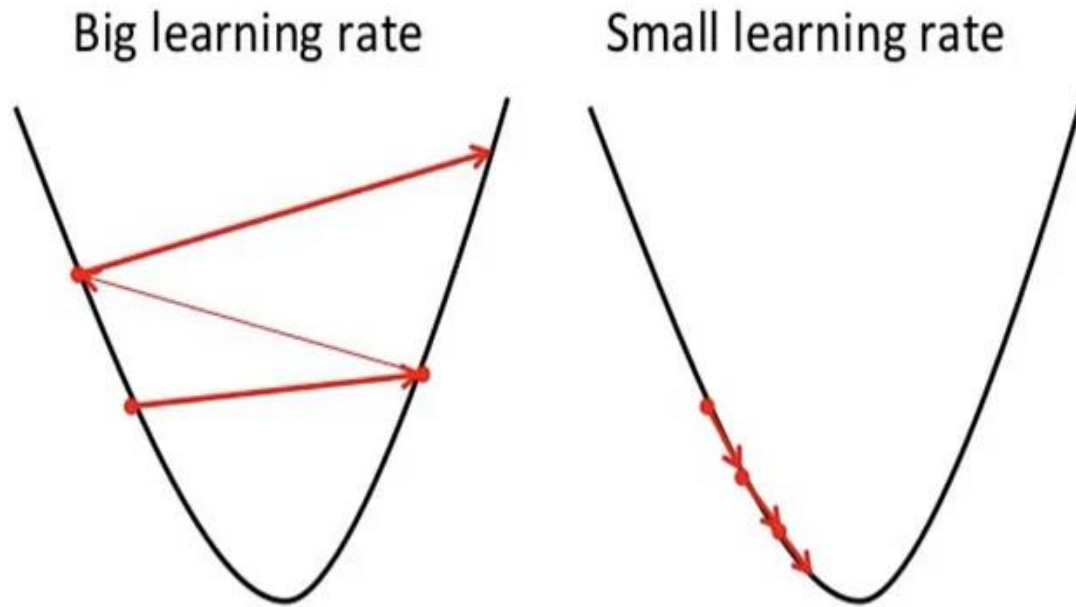
$$w_i = w_i - \alpha \nabla J(w_i) = w_i - \alpha \left(\frac{\partial J}{\partial w_i} \right)$$

- Repeat until slope = 0



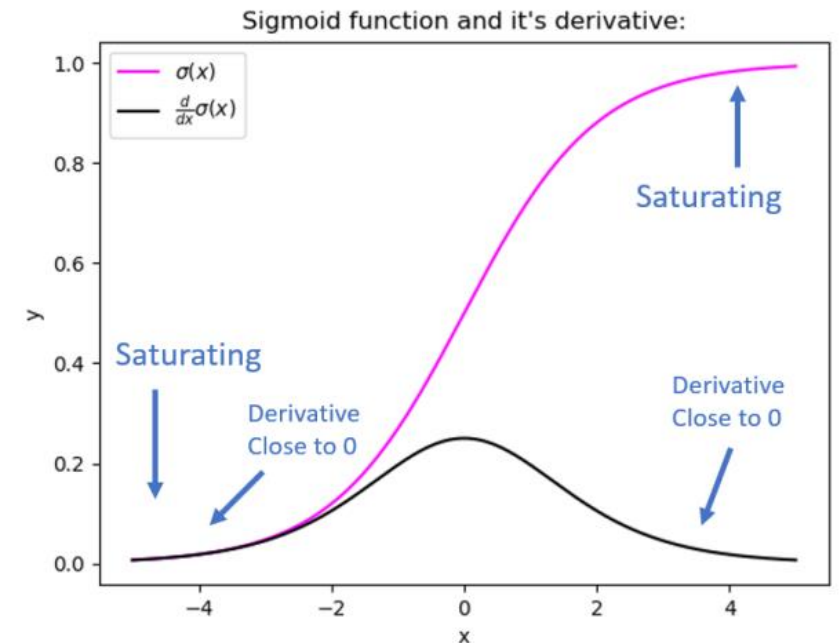
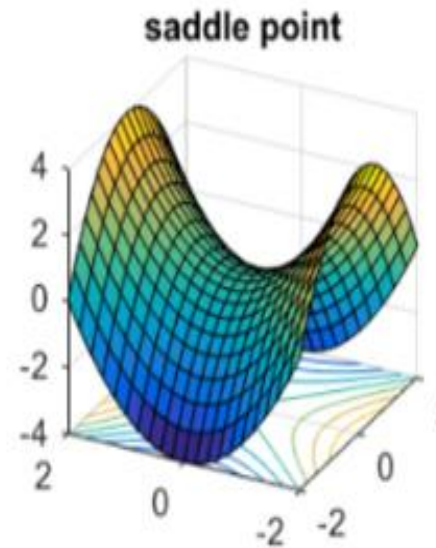
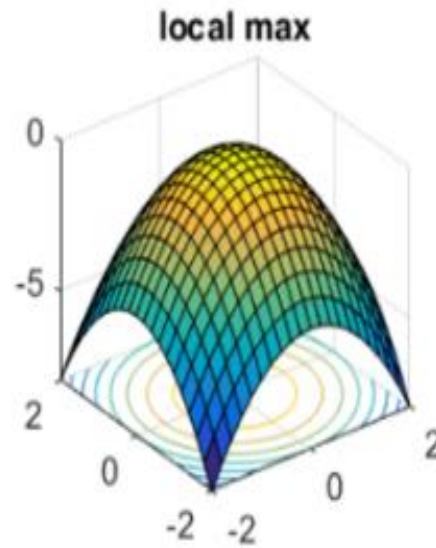
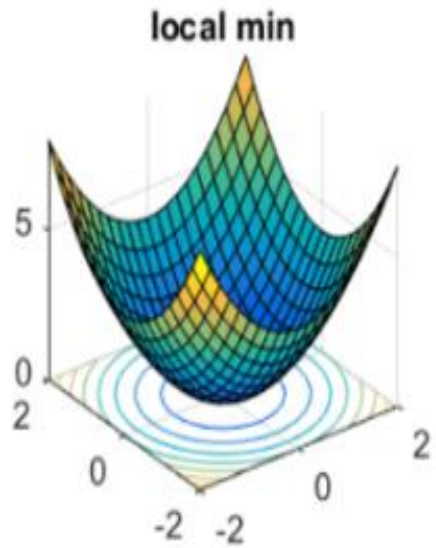
Example : Linear model – Learning rate

- Learning rate must be chosen wisely as:
 1. if it is too small, then the model will take some time to learn.
 2. if it is too large, model will converge as our pointer will shoot and we'll not be able to get to minima.



Issues that might occur

- When training a deep neural network with gradient descent and backpropagation, calculate the partial derivatives by moving from the final output layer to the initial layer.
- With the chain rule, layers that are deeper in the network applying continuous matrix multiplications to compute their derivatives. E.g.,
$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial A_1} * \frac{\partial A_1}{\partial Z_1} * \frac{\partial Z_1}{\partial w_1}$$
- Due to this process, **vanishing** gradient, **exploding** gradient and **saddle** point occurs.

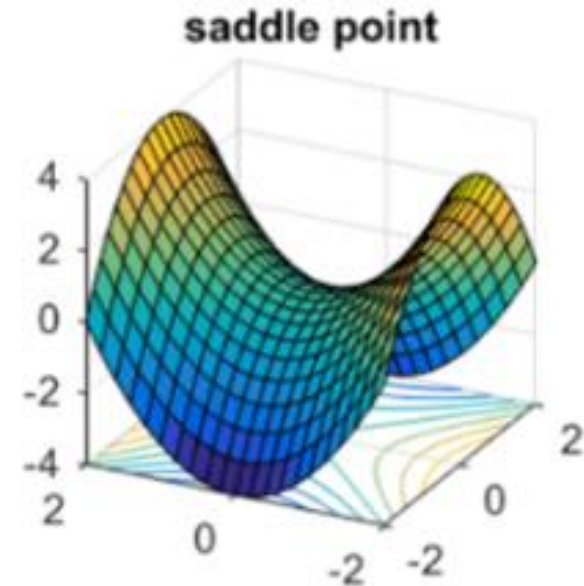


Saddle point

- Saddle (minimax) point is a point on the surface of the graph of a function where the slopes (derivatives) in orthogonal directions are all zero (a critical point), but which is not a local extremum of the function.

$$\frac{\partial J}{\partial w_1} = \frac{\partial J}{\partial A_1} * \frac{\partial A_1}{\partial Z_1} * \frac{\partial Z_1}{\partial w_1}$$

- Saddle point injects confusion into the learning process.
- Learning of the model becomes slow.
- It means that the crucial point achieved is the maximum cost value.
- This saddle point gets the focus when the gradient descent works in multi dimensions.



Avoid Exploding and Vanishing gradient problems

Vanishing gradients

- ReLU as the activation function
- Reduce the model complexity (Reduce the number of layers)
- Weight initializer with variance
- Better optimizer with a well-tuned learning rate

Exploding gradients

- Gradients clipping
- Proper weight initializer
- L2 norm regularization

Limitations of gradient computations

- For a good generalization model should have a **large training set**, which comes with a **huge computational cost** i.e., as the training set grows to billions of examples, the time taken to take a **single gradient step** becomes long.

Optimizers

- Optimizers are algorithms or methods used to change the attributes of neural network such as weights and learning rate to reduce the losses.

Epoch - The number of times the algorithm runs on the whole training dataset.

Batch - Number of samples considered for updating the model parameters.

Hypothesis: $y' = h_w(x) = w_0 + w_1(x)$

Parameters: w_0, w_1

Cost Function: $J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (y_i - y'_i)^2$

Goal: *minimize*
 w_0, w_1 $J(w_0, w_1)$

Various of Optimizers

1. Gradient Descent
2. Stochastic Gradient Descent
3. Stochastic Gradient descent with momentum
4. Mini-Batch Gradient Descent
5. Adagrad
6. RMSProp
7. AdaDelta
8. Adam

Batch Gradient Descent

Cost Function: $J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (y_i - y'_i)^2$

Goal: $\underset{w_0, w_1}{\text{minimize}} J(w_0, w_1)$

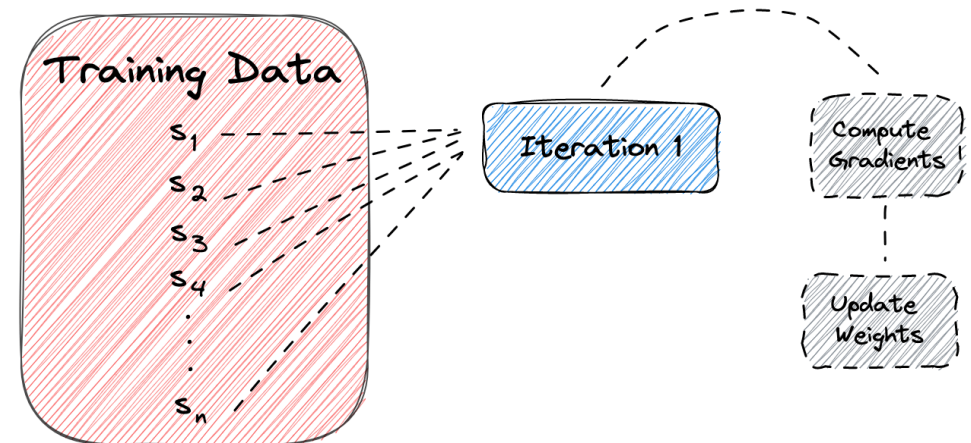
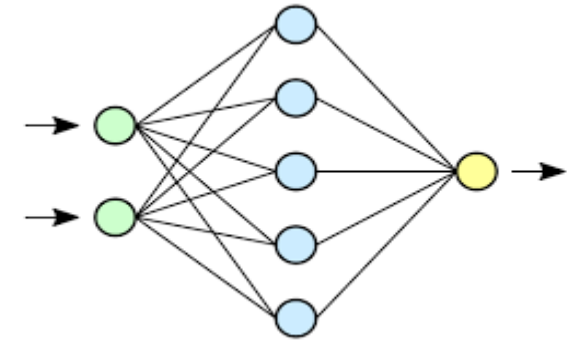
- **Batch Gradient Descent** uses **entire training dataset** in a **single iteration** of the algorithm.
- So, first pass all the training examples into the network, and compute the **gradient** of the loss function for each sample.
- Then, take the **average of the gradients** and update the parameters using the computed average.

Working Principle

- Initialize the random values to weight W and bias b parameters.
- Compute the cost J .
- Compute the gradient (∇W_i) of the cost function w.r.t each parameter.
- It moves towards the lower weight and updates the value of the coefficients i.e., weights.
- The process repeats until the local minimum is reached.
- Repeat until convergence {

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^m (y_i - y'_i)^2 * x_j^{(i)}$$

}

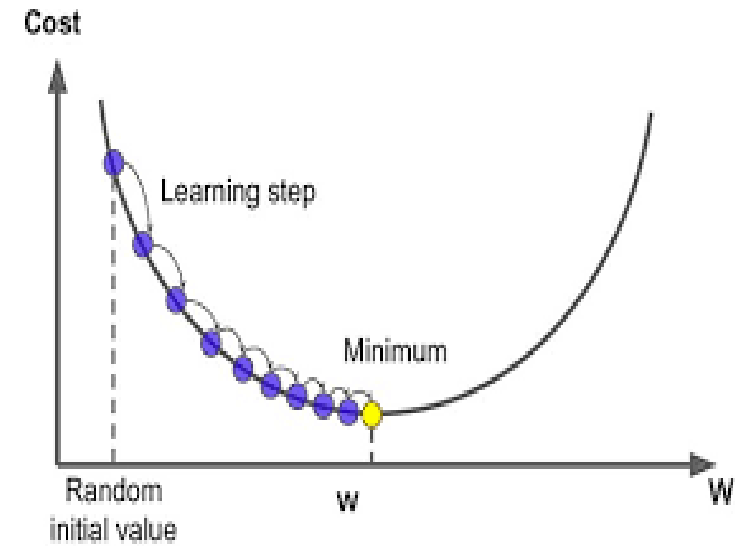
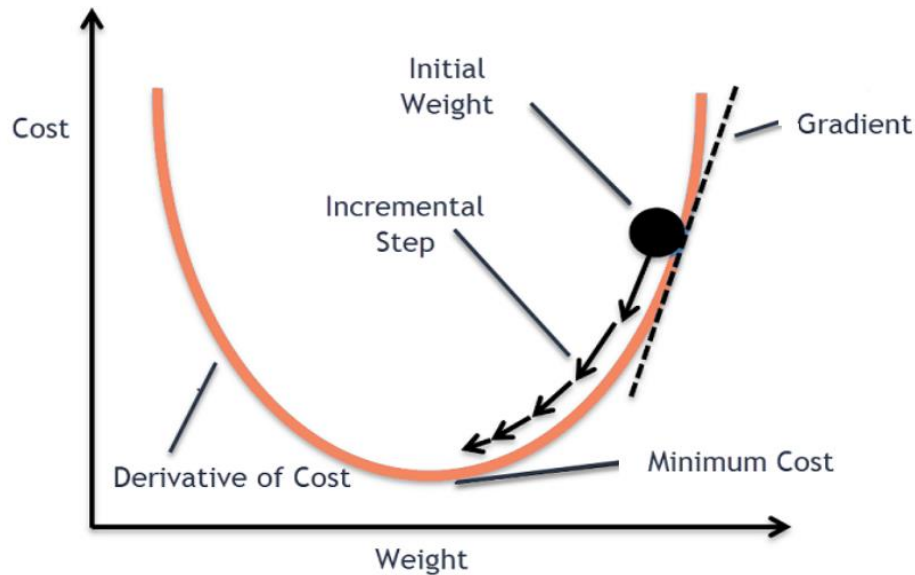


Batch Gradient Descent

- Repeat until convergence {

$$w_j = w_j - \alpha \frac{1}{m} \sum_{i=1}^m (y_i - y'_i)^2 * x_j^{(i)}$$

}



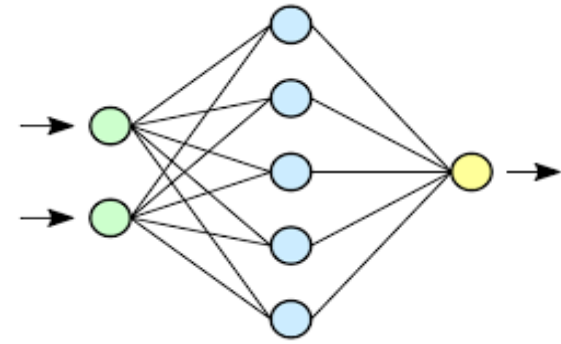
- Batch Gradient Descent is expensive to calculate the gradients for large dataset.
- Gradient descent works well for convex functions, but it consumes long time for nonconvex functions.

Stochastic Gradient Descent

$$\text{Cost Function: } J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (y_i - y'_i)^2$$

Goal: $\underset{w_0, w_1}{\text{minimize}} J(w_0, w_1)$

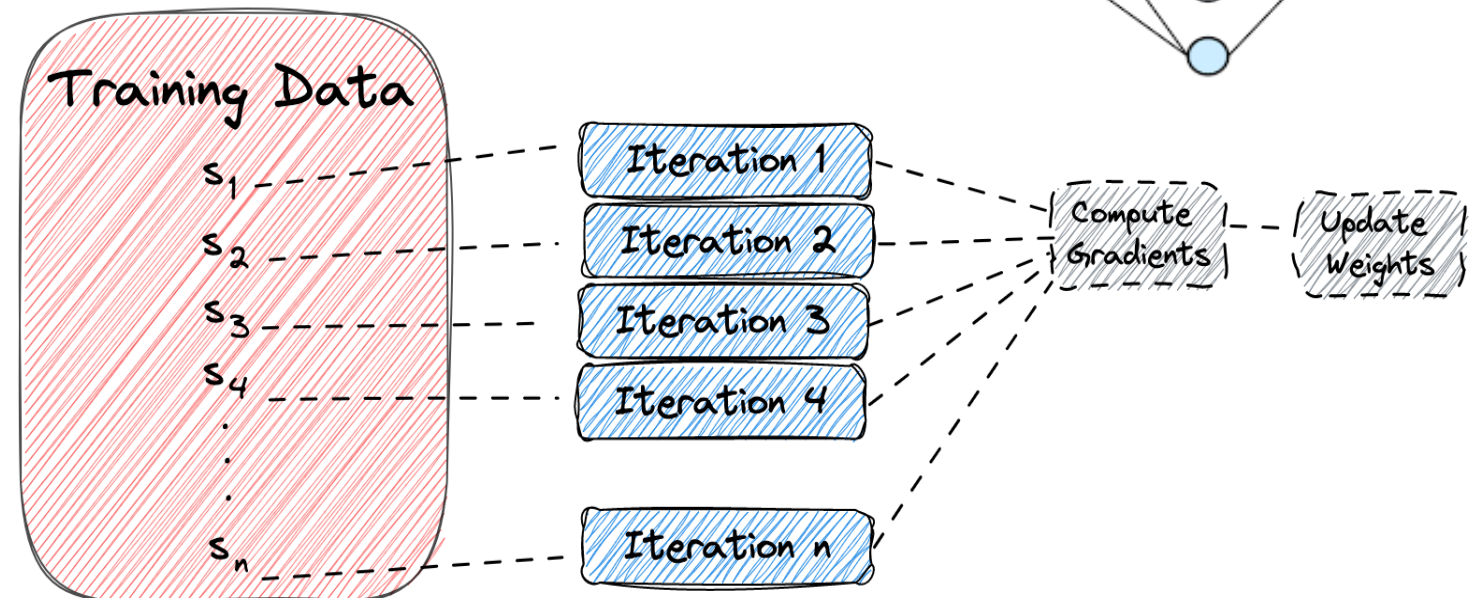
- **Stochastic gradient descent** is a variant of gradient descent.
- The process simply **takes only one random training example**, iterates and then improves before moving to the next random training example.
- It leads to frequent updates of W .
- Since only need to hold one training example, easier to store in memory.
- Due to frequent updates, it is treated as a noisy gradient.
- SGD saves time, and computing space.



- Repeat Until Convergence{
for i in range (m):

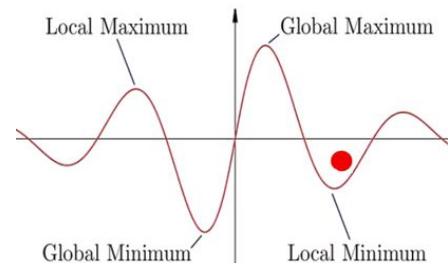
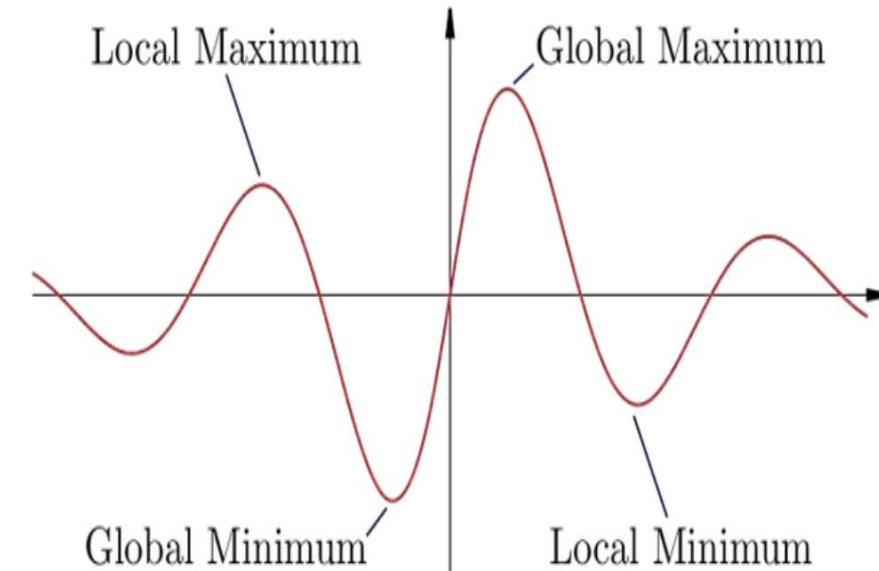
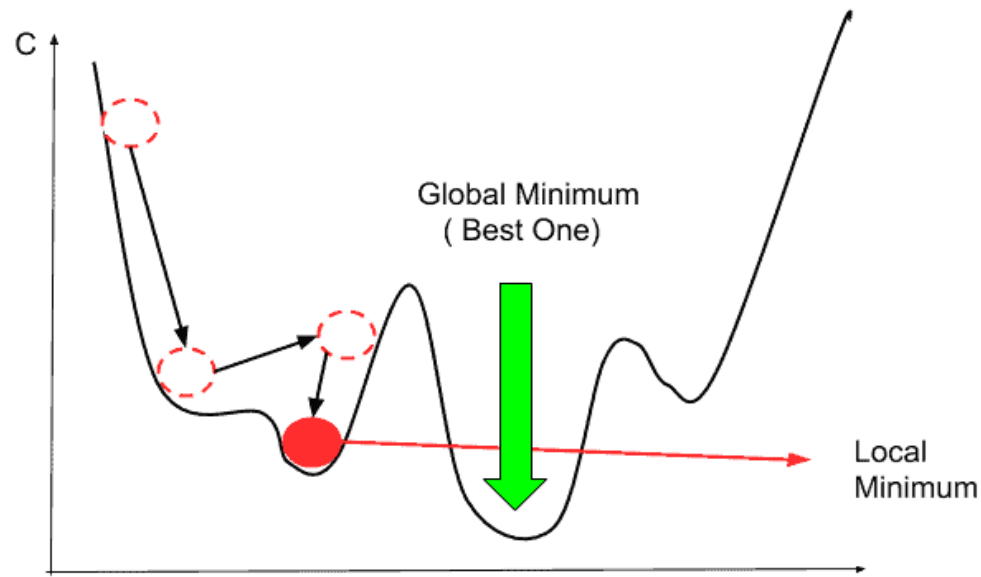
$$w_j = w_j - \alpha(y'_i - y_i) * x_j^{(i)}$$

}



Stochastic Gradient Descent

- If algorithm uses a noisy gradient, a gradient that points in different directions, rather than in one direction. This is called stochastic gradient descent (SGD).
- SGD avoids the local minima and **reach better (global) minima** due to adding noisy gradient.



Challenges in Batch Gradient Descent & SGD

- Batch Gradient Descent can be used for smoother curves.
- Batch Gradient Descent converges directly to minima.
- SGD can be used when the dataset is large. SGD converges faster for larger datasets. But, since SGD uses only one example at a time, we cannot implement the vectorized implementation on it. This leads to slow down the computations.
- To handle this issue, a mixture of Batch Gradient Descent and SGD is used.

Mini-Batch Gradient Descent

$$\text{Cost Function: } J(w_0, w_1) = \frac{1}{2m} \sum_{i=1}^m (y_i - y'_i)^2$$

Goal: $\underset{w_0, w_1}{\text{minimize}} J(w_0, w_1)$

- Mini-batch gradient descent **divides** the entire dataset into **randomly picked batches** and optimizes it.
- The mini-batch is a **fixed number** of training examples that is **less than** the actual dataset.
- So, the network is trained on a different **batch of samples** in each iteration, compute the gradient for each batch, and update the parameters.
- Repeat this until all samples of the dataset are used.

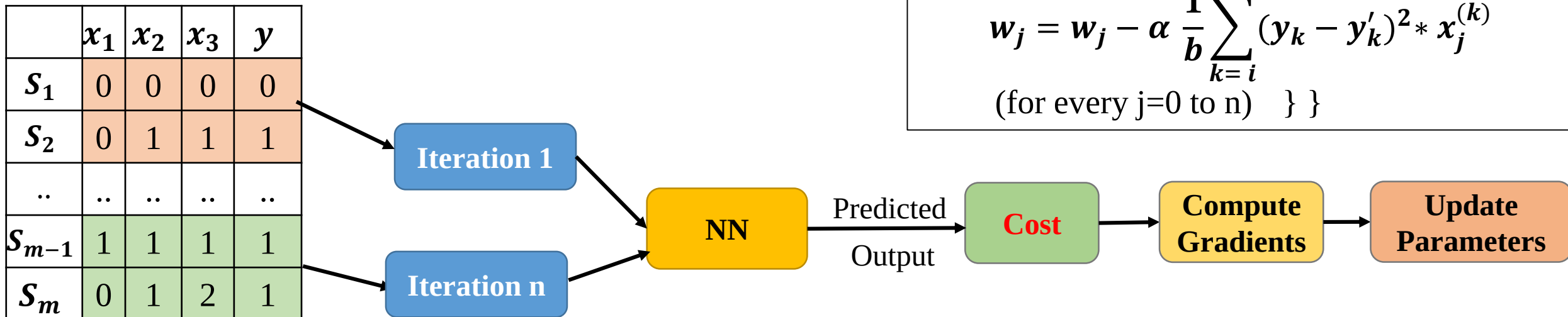
batch **$b = 10$** , $m = 1000$,

Repeat {

for $I = 1, 11, 21, 32 \dots 991$ {

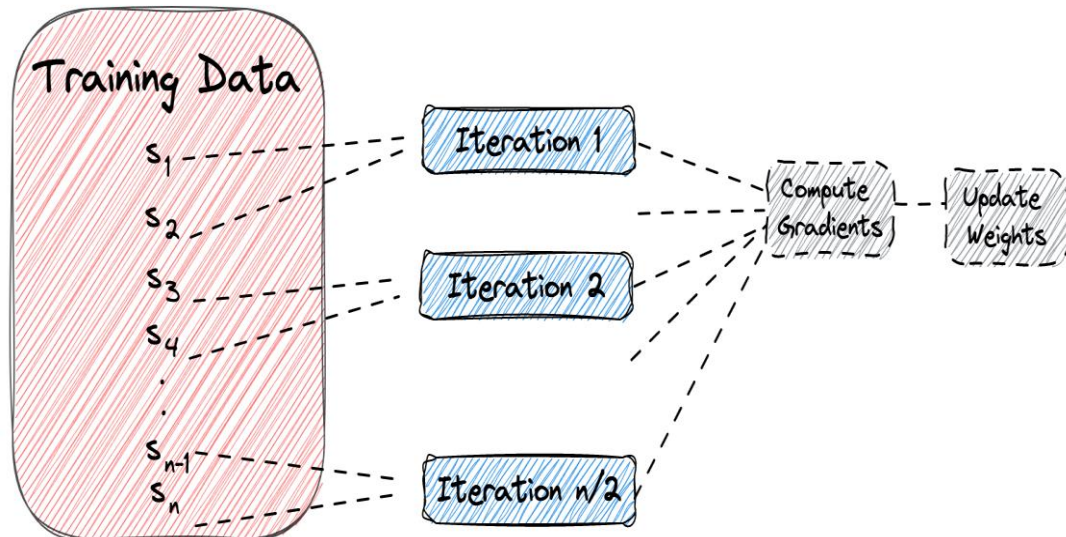
$$w_j = w_j - \alpha \frac{1}{b} \sum_{k=i}^{i+9} (y_k - y'_k)^2 * x_j^{(k)}$$

(for every $j=0$ to n) } }

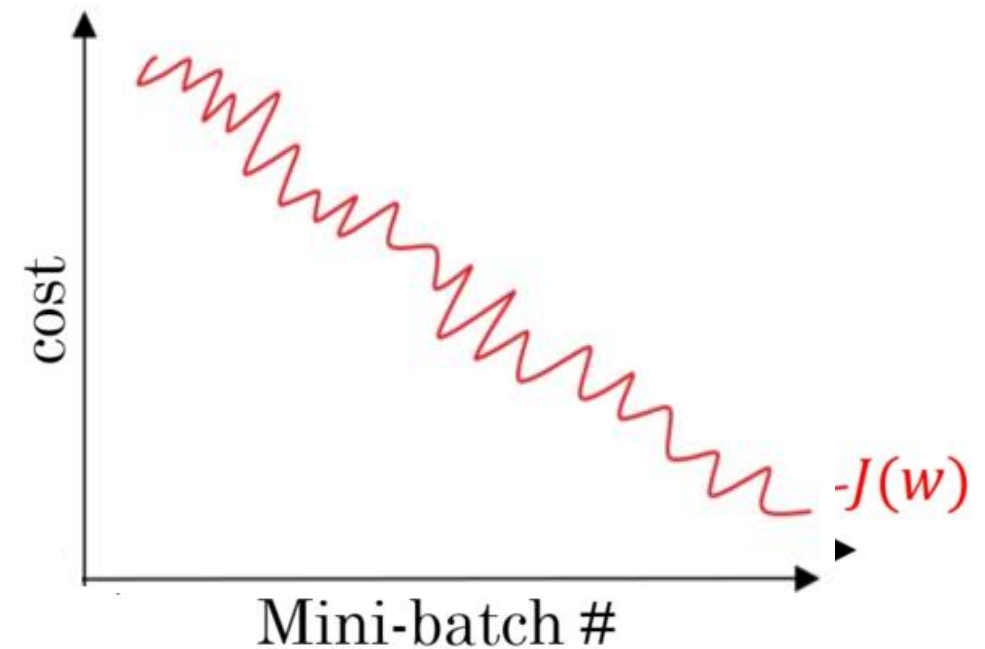


Mini-Batch Gradient Descent

- The average cost over the epochs in mini-batch gradient descent **fluctuates** due to averaging a small number of examples (mini-batch) at a time.
- It updates the parameters frequently as well as using vectorized implementation for faster computations.
- Mini-batch requires the configuration of an addi

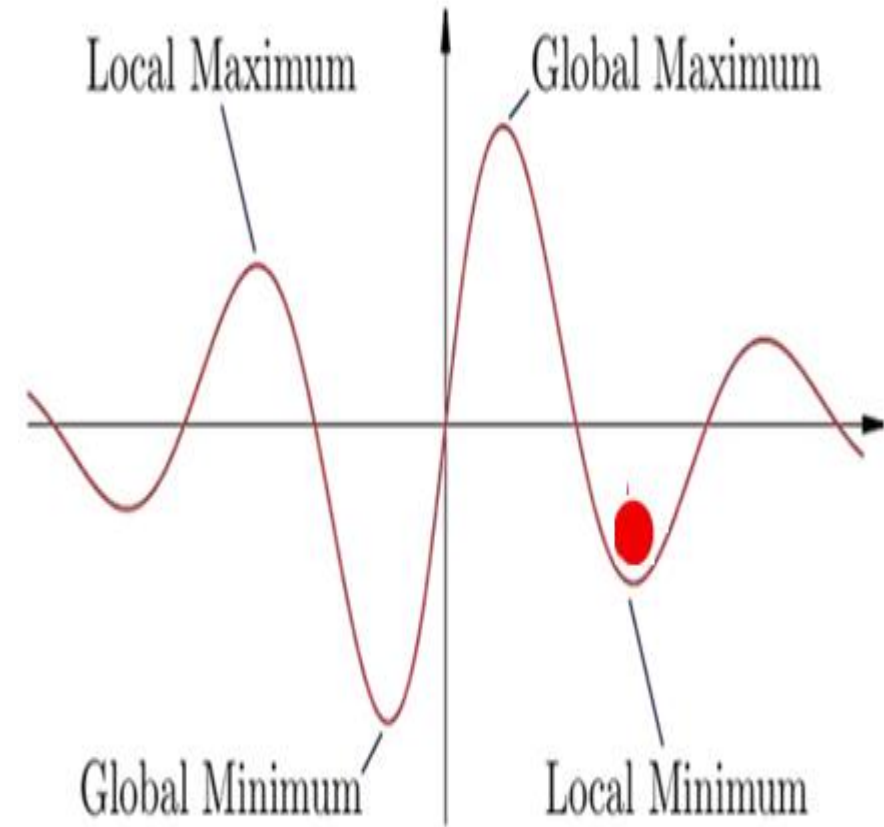


Mini-batch gradient descent



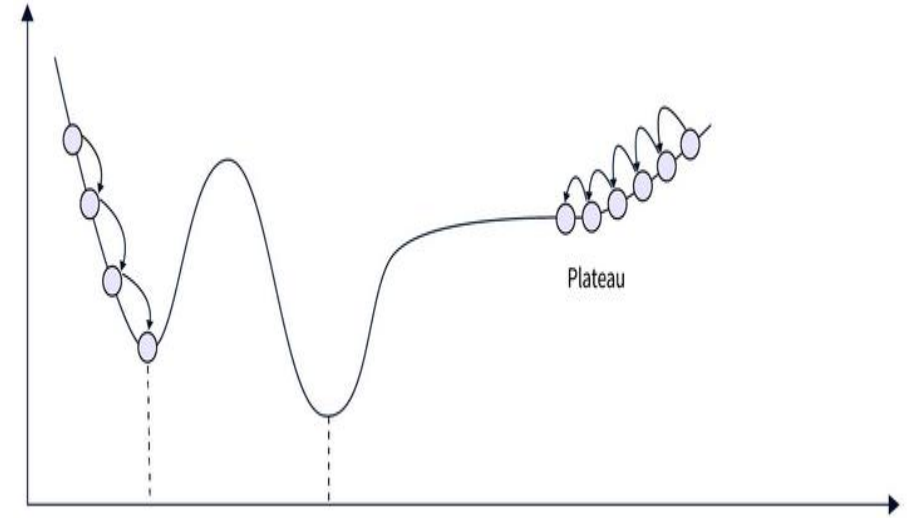
Challenges in Gradient Descent

- Issue with GD is accidentally getting **stuck in local minima**, where loss can still be HUGE.
- Gradient descent takes **more time** to pass through the gentle slope.
- Because at gentle slope gradient is very small so update becomes slow.
- To solve this problem, the **momentum** will add to the gradient descent.
- Saddle Points:
 - A saddle point is a point in the cost function where one dimension has a higher value than the surrounding points, and the other has a lower value. Gradient descent can get stuck at these points because the gradients in one direction point towards a lower value, while those in the other direction point towards a higher value.



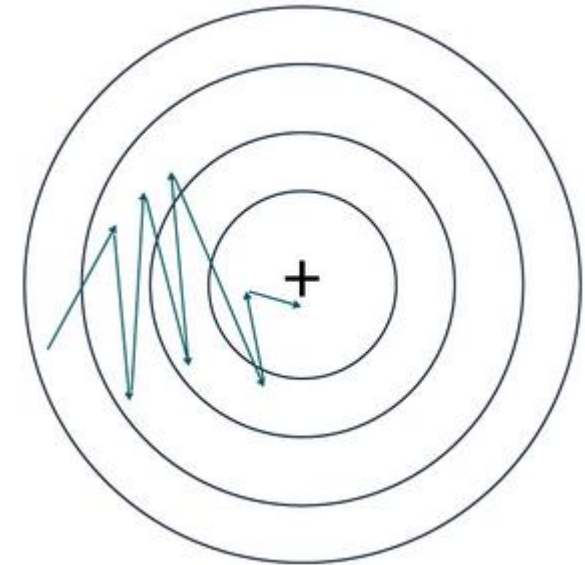
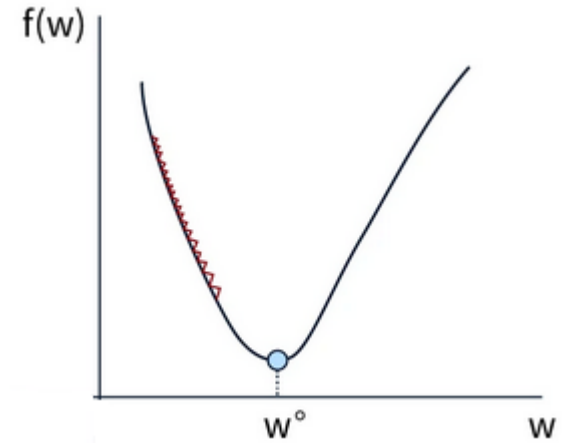
Challenges in Gradient Descent

- Plateaus:
- A plateau is a region in the cost function where the gradients are very small or close to zero. This can cause gradient descent to take a long time or not converge.
- Oscillations:
- Oscillations occur when the learning rate is too high, causing the algorithm to overshoot the minimum and oscillate back and forth.



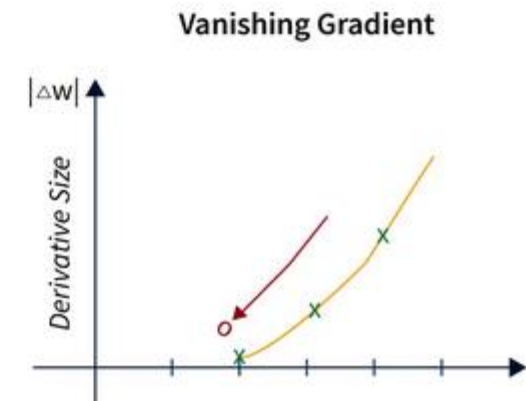
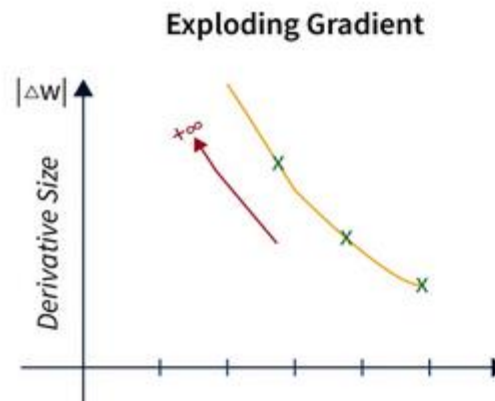
Challenges in Gradient Descent

- Slow convergence:
- Gradient descent can converge very slowly when the cost function is complex or has many local minima. This means the algorithm may take a long time to find the global minimum.
- Stochasticity:
- In stochastic gradient descent, the cost function is evaluated at random samples from the data set. This introduces randomness into the algorithm, making converging to a global minimum more difficult.



Challenges in Gradient Descent

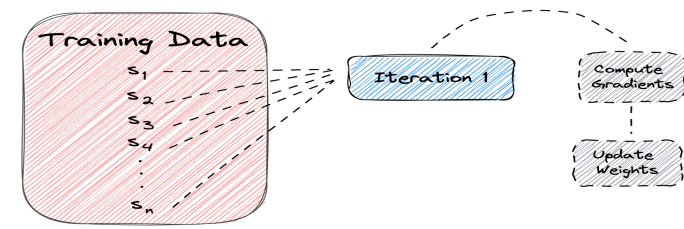
- Vanishing or Exploding Gradients:
- Deep neural networks with many layers can suffer from vanishing or exploding gradients. This occurs when the gradients become very small or large, respectively, as they are backpropagated through the layers. This can make it difficult for the algorithm to update the weights and biases.



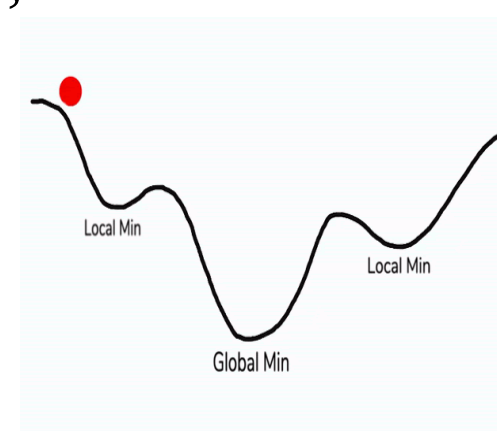
Momentum Fix the Problems

- It works by adding a fraction of the previous weight update to the current weight update so that the optimization algorithm can build momentum as it descends the loss function. This can help the algorithm escape from local minima and saddle points and can also help the algorithm converge faster by avoiding oscillations.
- Momentum helps to,
 - Escape local minima and saddle points
 - Aids in faster convergence by reducing oscillations
 - Smooths out weight updates for stability
 - Reduces model complexity and prevents overfitting
 - Can be used in combination with other optimization algorithms for improved performance.

Momentum-based Gradient Optimizer

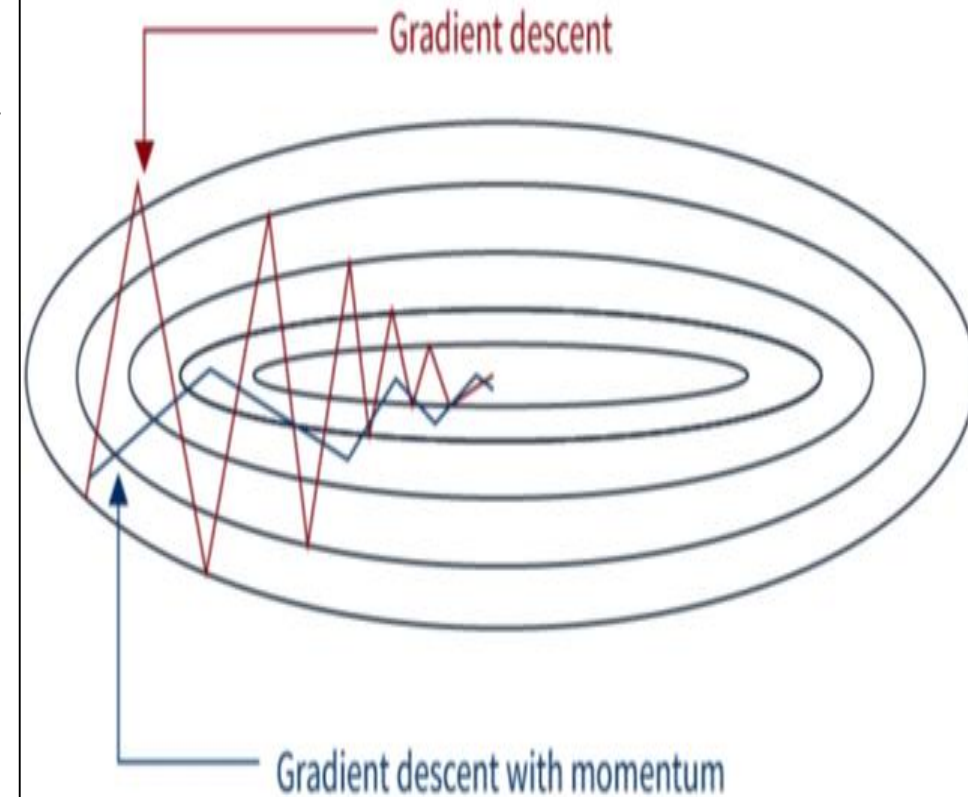
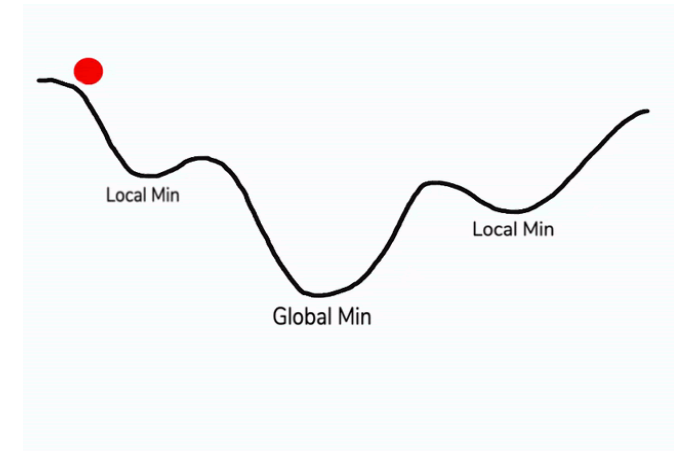


- It is an extension of Batch Gradient Descent.
- Momentum is a strategy for **accelerating** the convergence of the optimization process by including a momentum element in the update rule.
- Momentum considers **previous** step weights gradient changes when updating **current** weights.
- This momentum factor assists the optimizer in **continuing** to go in the **same direction** of the search space even if the gradient changes direction **or** becomes zero.
- It overcome the oscillations of noisy gradients and coast across flat spots of the search space.
- Instead of stopping when the gradient is 0 at the first local minimum, momentum will continue to move forward because it considers the prior steep of the slope.



Momentum-based Gradient Optimizer

- $v(t) = \beta * v(t - 1) + (1 - \beta) \nabla(w)$; $\nabla(w) = \frac{\partial J}{\partial w_t}$
- $w = w - \alpha * v(t)$
- v – velocity, β is momentum range is 0 to 1 (typically 0.9).
- The optimizer calculates the gradient of the cost function at each iteration, and updates the momentum term as the **exponentially weighted moving average** of the previous gradients.
- It helps the algorithm to escape from local minima, and saddle points.
- Also makes the algorithm to converge faster by avoiding oscillations.



Adagrad (Adaptive Gradient) optimizer

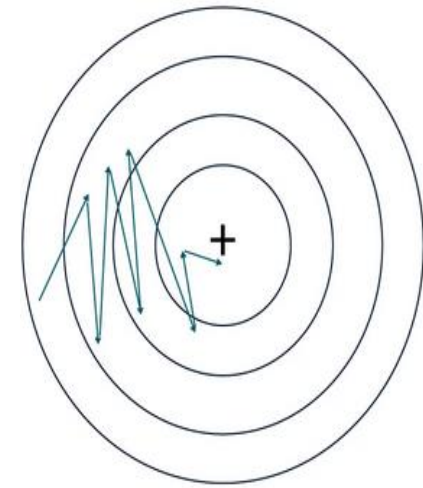
- Gradient descent follows the **negative gradient** of an objective function to locate the minimum of the function.

Limitation of gradient descent

- **Gradient descent** uses the **same step size** (learning rate) for each parameter or input feature.
- This leads to have different amounts of curvature in different dimensions.
- So, algorithm needs a **different sized step (learning rate)** to **each parameter**.

Need of Adagrad:

- AdaGrad is an extension of the gradient descent optimization.
- Adagrad Optimizer allows **different learning rate** (α) for **each weight** (w).
- In the real-world dataset, some input features are sparse (i.e., most of the features are zero) and some input features are dense (most of the features are non-zero).
- In this scenario, the **same learning rate** (α) for all the weights is **not good** for optimization.



Adagrad (Adaptive Gradient) optimizer

- So, Adagrad algorithm **adapts** the learning rate to the parameters. It is using **low learning rates** for parameters associated with **frequently** occurring features, and **high learning rates** for parameters associated with **rare features**.
- This is well suitable for the sparse data.

E.g.,

- During training some parameters might reached **near** to **converging stage** where only **fine adjustment** is required, but some parameters **need to be adjusted significantly** due to the small number of matching samples i.e., **far away** from convergence.
- Weight update rule for Adagrad:
 - $\mathbf{w}_t = \mathbf{w}_{t-1} - \eta_t * \left(\frac{\partial J}{\partial \mathbf{w}_{(t-1)}} \right)$
 - $\eta_t = \frac{\eta}{\sqrt{\alpha_t + \epsilon}}$; α_t - different learning rates for **each weight** at **each iteration**.
 - ϵ – *small positive constant to avoid ‘division by 0’ error* when $\alpha_t \rightarrow 0$, $\epsilon = 10^{-8}$.
 - if α_t becomes 0 then η_t will become zero which lead to **small convergence**, i.e., no change in w .

Adagrad (Adaptive Gradient) optimizer

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial J}{\partial w_{(t-1)}} \right)^2$$

- $\frac{\partial J}{\partial w_{(t-1)}}$ is derivative of loss with respect to weight and $\left(\frac{\partial J}{\partial w_{(t-1)}} \right)^2$ will always be positive since its a square term. So, α_t is always positive due to squaring the gradients, this implies that $\alpha_t \geq \alpha_{(t-1)}$.
- From this, α_t and η_t is inversely proportional to one another. $\eta_t = \frac{\eta}{\sqrt{\alpha_t} + \epsilon}$
- This implies that as α_t will increase, η_t will decrease. So, the number of iterations will increase, the learning rate will reduce adaptively.
- Hence, **no need** to select the learning rate **manually**.

Advantages of Adagrad:

- **No manual** tuning of the learning rate required.
- Faster convergence and more reliable.

Disadvantage

- α_t can become large increases the number of iterations. Due to this η_t will decrease at the larger rate. This will make the old weight almost equal to the new weight which may lead to slow convergence.

Root Mean Squared Propagation (RMSProp) optimizer

Limitation of AdaGrad

- α_t can become large as the number of iterations will increase and due to this η_t will decrease at the larger rate. This will make the old weight almost equal to the new weight which may lead to slow convergence.
$$\eta_t = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

RMSProp

- RMSProp is an adaptive learning rate optimization algorithm.
- RMSProp is a variant of the Gradient Descent and AdaGrad algorithms that improves the convergence speed, and stability of the model training process.
- It uses moving average of the **squared gradients** [i.e., $(\nabla_w J(w_t))^2$] to scale (exponentially decay) the learning rate for each parameter.
- It makes the learning rate is **less than** a certain threshold.
- RMSProp is designed to dramatically reduce the computations in training neural networks.

Root Mean Squared Propagation (RMSProp) optimizer

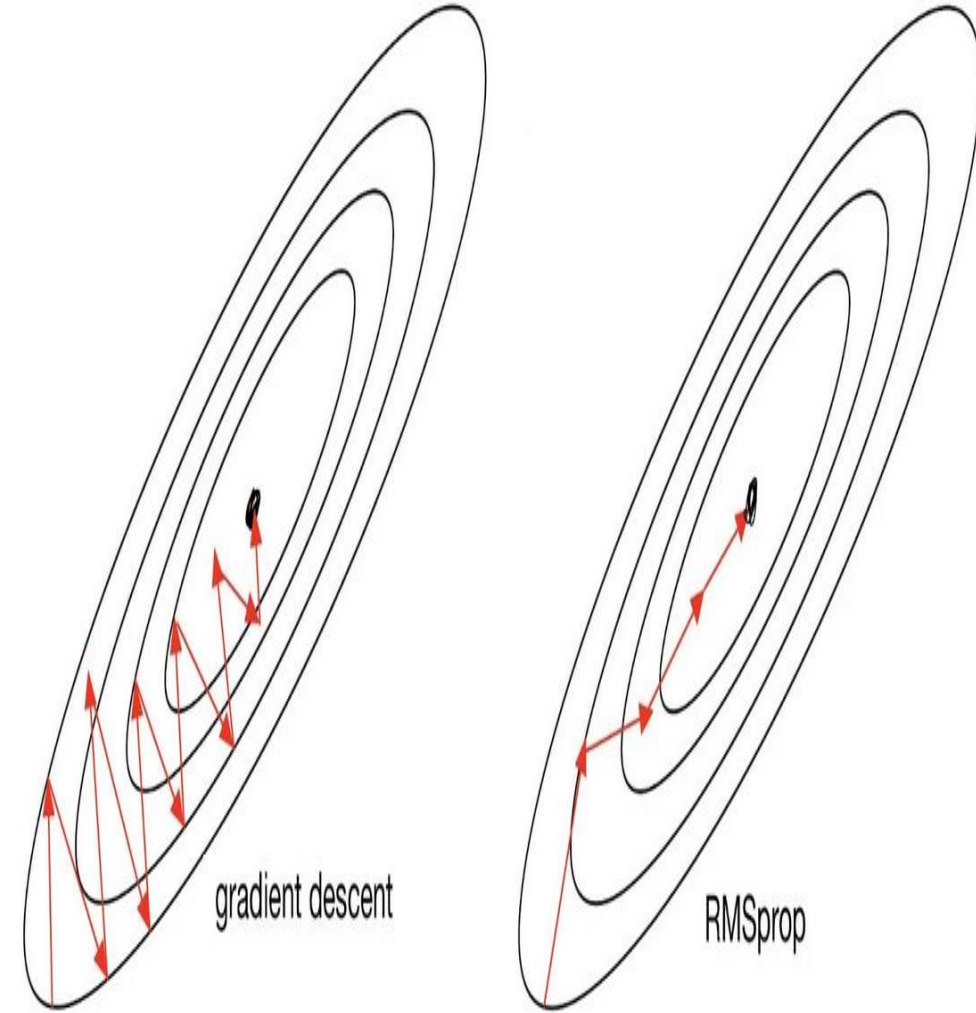
- This helps reduce the learning rate by **forgetting early gradients**, and **focus on the most recently** observed partial gradients during the progress of the search, overcoming the limitation of AdaGrad.
- It leads to smoothly adjust the learning rate for each parameter in the network, providing a better performance than Batch Gradient Descent.
- RMSprop algorithm utilizes **exponentially** weighted moving averages of squared gradients to update the parameters i.e.,
 $(\nabla_w J(w_t))^2$

Root Mean Squared Propagation (RMSProp) optimizer

- Initialize parameters:
 - Learning rate: α , **Exponential decay** rate for averaging β
 - Small constant for numerical stability ϵ , Initial parameter values w
- Initialize accumulated gradients (Exponentially weighted average):
 - Accumulated squared gradient for each parameter: $v_t = 0$
- Repeat until convergence or maximum iterations:
 - Compute the gradient of the objective function with respect to the parameters:
 - $g_t = \nabla_w J(w_t)$
 - Update the **exponentially weighted average** of the squared gradients:
 - $v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$
 - Update the parameters: $w_t = w_{t-1} - \alpha * \left(\frac{g_t}{\sqrt{\widehat{v}_t + \epsilon}} \right)$

Root Mean Squared Propagation (RMSProp) optimizer

- $\mathbf{g}_t$ is the gradient of the loss function with respect to the parameters at time t
- β is a **decay factor**, $\mathbf{v}_t$ is the exponentially weighted average of the squared gradients
- ϵ is a small constant to prevent division by zero
- This process is repeated for each parameter, and it helps adjust the learning rate for each parameter based on the historical gradients.
- The exponential moving average allows the algorithm to give **more importance** to **recent gradients** and dampen the effect of older gradients, providing stability during optimization.



Adam (ADaptive Moment estimation) optimizer

- Adam is an extension to stochastic gradient descent algorithm.
- Adam is a **first-order** derivative **optimization** algorithm
- It is using the stochastic gradient extensions of AdaGrad and RMSProp, to deal with ML & DL problems involving **large datasets** and **high-dimensional** parameter spaces.

Momentum:

- Momentum is used to accelerate the gradient descent algorithm by considering the **‘exponentially weighted average’** of the gradients.
- These averages makes the convergence towards the minima faster.
- An exponentially weighted moving average **reacts** more significantly to previous gradients changes.

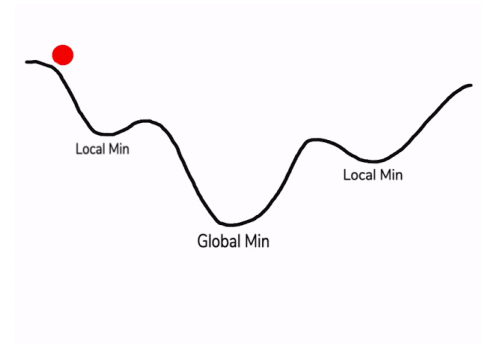
$$w_t = w_{t-1} - \alpha m_t$$
$$m_t = \beta \cdot m_{t-1} + (1 - \beta) \left(\frac{\partial J}{\partial w_t} \right)$$



Adam optimizer

- Algorithm control the **rate of gradient descent** to minimize the oscillation when it reaches the global minimum while taking big enough steps (step-size).

$$\mathbf{v}_t = \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \left(\frac{\partial J}{\partial \mathbf{w}_t} \right)^2$$



- It makes the algorithm to pass the **local minima hurdles** in a search space.
- Hence, combine the features of the above methods to reach the global minimum efficiently.

Mathematical Aspect of Adam Optimizer

- First momentum** update $\mathbf{m}_t = \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \left(\frac{\partial J}{\partial \mathbf{w}_t} \right);$
- Second momentum** update $\mathbf{v}_t = \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \left(\frac{\partial J}{\partial \mathbf{w}_t} \right)^2$
- β_1 & β_2 - exponential decay rates of average of gradients ($\beta_1 = 0.9$ & $\beta_2 = 0.999$). These are estimating the moments.
- Exponential Decay - learning rate schedule where decay (decrease) the α with more iterations using an exponential function: $\alpha = \alpha * e^{-kt}$, k - decay rate, t – iteration number

Adam optimizer

- Since $\mathbf{m}_t$ and $\mathbf{v}_t$ have both initialized as 0, it is observed that they gain a tendency to be ‘biased towards 0’ as both β_1 & $\beta_2 \approx 1$.
- This Optimizer fixes this problem by computing ‘bias-corrected’ $\mathbf{m}_t$ and $\mathbf{v}_t$. This controls the weights while reaching the global minimum to **prevent high oscillations** when near it.

$$\widehat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t} \quad ; \quad \widehat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}$$

- Intuitively, learning is adapting to the gradient descent after every iteration so that it remains controlled, and unbiased throughout the training, hence the name Adam.
- Now, instead of our normal weight parameters $\mathbf{m}_t$ and $\mathbf{v}_t$, we take the bias-corrected weight parameters $\widehat{\mathbf{m}}_t$ and $\widehat{\mathbf{v}}_t$.
- Use these into general equation as $\mathbf{w}_t = \mathbf{w}_{t-1} - \widehat{\mathbf{m}}_t \left(\frac{\alpha}{\sqrt{\widehat{\mathbf{v}}_t} + \epsilon} \right)$
- ϵ –small positive constant to avoid ‘division by 0’ error when $\mathbf{v}_t \rightarrow 0$, $\epsilon=10^{-8}$

Adam optimizer

- β_1 & β_2 = decay rates of average of momentum and RMS Prop methods ($\beta_1=0.9$, $\beta_2=0.999$)
- w_0 = initial parameter vector, $f(w)$ = stochastic objective function
- Initialize $m_0 = 0$ // First moment vector
- $v_0 = 0$ // Second moment vector
- $t = 0$ // timestep
- Repeat {

$t = t+1$

$g_t = \nabla(w) \cdot f_t(w_{t-1})$ // gradient at t w.r.t stochastic objective function

$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$ //update biased first moment estimate

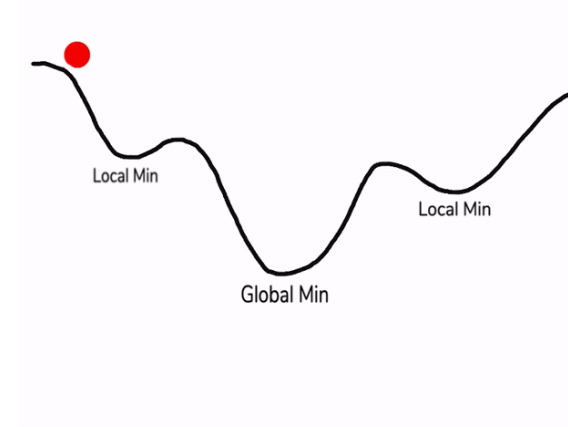
$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$ //update biased second raw moment estimate

$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$ // bias-corrected first moment estimate

$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$ // bias-corrected second raw moment estimate

$w_t = w_{t-1} - \hat{m}_t \left(\frac{\alpha}{\sqrt{\hat{v}_t} + \epsilon} \right)$ //update the parameters

- until w_t reach convergence.



Batch GD vs SGD vs Mini Batch

Batch Descent	Stochastic Descent	Mini-Batch Descent
Uses all training samples while training the dataset and only after the complete cycle makes the required update in the training dataset.	It uses one random training sample and makes the required update in the training dataset.	Divides the dataset into batches and, after completion of every batch makes the required update in the dataset.
Requires too much time and computational space for one epoch.	Faster than Batch descent and requires moderate computational space.	It is the fastest and requires minimal computational space.
Best to use for small training datasets	Best for training big datasets with less computational	Best for large training datasets.

PARAMETERS	BATCH GD ALGORITHM	MINI BATCH ALGORITHM	STOCHASTIC GD ALGORITHM
ACCURACY	HIGH	MODERATE	LOW
TIME CONSUMING	MORE	MODERATE	LESS

Key points (Epoch, batch, sample)

- Example: Assume there are 70 000 samples. Let's arbitrarily choose a batch size of 100 and a number of epochs equal to 500.

$$\#Batches = \frac{\#Samples}{Batch\ size} = \frac{70000}{100} = 700$$

- One epoch will represent a 100 update of the model's parameters. 500 epochs, the dataset takes 500 times. It represents a total of 50 000 batches.

$$Total\ \#Batches = \#Epochs \times Batch\ size = 500 \times 100 = 50000$$

<https://www.analyticsvidhya.com/blog/2021/03/variants-of-gradient-descent-algorithm/>

Issues in GD

<https://www.scaler.com/topics/momentum-based-gradient-descent/>

Example

<https://www.geeksforgeeks.org/intuition-of-adam-optimizer/>

<https://optimization.cbe.cornell.edu/index.php?title=Adam>

brief

<https://towardsai.net/p/l/impact-of-optimizers-in-image-classifiers>